Programación Genética

Prof. Eddy Ramírez Jiménez

21 de agosto de 2025

1. Motivación

La inteligencia artificial, nos ofrece alternativas a la forma en como programamos, incluso hay algunos pseudo-programadores que en lugar de hacer un análisis cuidadoso de un problema para ofrecer una solución viable, empiezan a escribir código sin entender completamente el problema en cuestión, a veces buscan un código de internet que copian y empiezan a hacer un "*simulated annealing*" de cambios aleatorios hasta que el código se ajuste a lo que querían inicialmente. Aunque esto no pueda considerarse necesariamente "inteligente", esto sí puede ser automatizado, es decir, se puede hacer un programa que "batee" código hasta que salga algo que parezca que funciona. Desde luego, todo esto es superior a considerar que validar una entrada es más importante que desarrollar un algoritmo.

Su tarea, consiste en hacer un programa que sea capaz de generar una función que aproxime puntos en un ambiente tridimensional, esta función recibe pares de números (x, y) y genera un número (z) . Todo esto principalmente en cuanto a poder darle una aplicabilidad real a las ventajas que ofrece un lenguaje interpretado como Scheme, donde los programas pueden verse como árboles y por ende su representación y ejecución está muy entrelazada.

2. Objetivos Formativos

2.1. Objetivo(s) del curso que respalda el proyecto

- Al finalizar el curso el estudiante debe ser capaz de aplicar diferentes enfoques y estrategias en la creación de programas computacionales que simulen comportamientos observados en la naturaleza.
- Aprender un lenguaje ejemplo para cada uno de los cuatro paradigmas principales de programación, así como conocer varios otros lenguajes para representar conceptos que no tengan los lenguajes ejemplo.

- Ofrecer criterios para comparar, evaluar y seleccionar lenguajes de programación.

- Dar criterios para discernir cuál es el lenguaje de programación más adecuado para un problema dado

3. Especificación del proyecto

Su proyecto consiste en que dados un conjunto de puntos dados en un archivo, (entre 2 y 2000 puntos) debe hacer un programa que genere una función que pase por todos estos puntos. Para ello se utilizará programación genética y se garantiza que todas las funciones "reales" que se usaron para generar esos puntos cumplen con las siguientes características:

- Sus operadores son binarios
- Su dominio es $\mathbb{R}^2$
- Su codominio es $\mathbb{R}$
- Los operadores que se emplearán serán $+, -, *, /, expt, log_a b$

Se selecciona Scheme porque en este lenguaje los códigos son árboles, lo que facilita el modelado de los individuos y a la vez permite manipularlos como tales, haciendo uso del *eval*, se pueden correr y medir qué tan cerca se encuentra un individuo de ser solución al problema. Es importante considerar que la división entre cero no está definida, por lo que sus programas deben validar esto (entre otras posibles funciones que pueden dar error), haciendo una función para atrapar esos casos.

Para resolver el problema, deberá realizar programación genética, es decir, deberá crear un número de funciones aleatorias, las cuales reciben todas dos parámetros, a los que llamaremos x y y, y se pondrán dentro de una expresión algebraica con valores y operadores aleatorios. Se sugiere que la población inicial sea diversa, tanto en cantidad de operaciones como en el número de dichas operaciones por función.

En cada generación de las poblaciones, su programa deberá mostrar gráficamente la mejor de las funciones, es decir, la que más se aproxima a los puntos destino. Se debe detener cuando encuentre una función que pasa por todos los puntos, es decir, cuando haya encontrado una solución. Se sugiere que en la pantalla se muestre la función de un color y los puntos de otro, para ir observando como se va aproximando la función a la solución y que se trabaje con elitismo. Esta opción se podrá apagar para que el programa trabaje más rápidamente, pero para la revisión, es muy necesaria y que permita explicar como es que se va aproximando.

4. Metodología

Su tarea debe constar de:

- Graficar funciones 3D en Racket (hay completa libertad de las bibliotecas gráficas a utilizar)
- Generar poblaciones aleatorias
- Generar función de fitness (función de evaluación)
- Programar algún mecanismo de selección y cruce de candidatos (ojalá con elitismo y recordar que un individuo no puede repetirse en una población).
- Generar algún mecanismo de mutación

5. Rúbrica

Producto esperado	Porcentaje
Gráfica en 3D con puntos en dos colores durante la ejecución	20 %
Generación de funciones aleatorias y genéticamente diversas	10 %
Selección, cruce y mutación de acuerdo con criterios válidos	10 %
Funcionamiento con f arbitraria	80 %
Máximo TOTAL permitido	120 %

Nota los puntos de diferencia se calcularán con el valor absoluto de las diferencias entre la función generada y los puntos dados. No habrá puntos extra, si se resuelven todas las funciones, se colocará un ciento veinte como nota máxima.

6. Estimación de tiempo

- Se espera que el programa se realice rápidamente y que trabaje por varias horas hasta poder dar con las respuestas. Considerar que es muy posible que no haya una respuesta única que sea considerada como correcta. Se recomienda que utilicen cada cierto tiempo, un archivo para guardar los mejores individuos cada n generaciones.
- Salvo para las bibliotecas gráficas o funciones primitivas de Scheme, todo lo demás debe ser programado por los estudiantes. Se considerará plagio cualquier uso de código externo que no contenga los comentarios de *Dominio* y *Codominio*.

7. Aspecto Generales

- Formato de entrega: Archivo con el código fuente al correo del profesor.
- Canal de entrega: Revisión contra entrega el día de la revisión en horario de clase.

8. Funciones por encontrar

En esta sección se indican las funciones f_1, f_2, f_3 que son las que suman puntos en la evaluación, sin embargo, se dejan algunas funciones más simples con las definiciones en el nombre, para que sean tomadas como pruebas de concepto en la ejecución del programa, se invita a los estudiantes a hacer funciones propias.

8.1. f_1

(19 15 29346.35851851852)
(2 9 1.0534979423868314)
(2 2 21.333333333333332)
(6 14 105.79591836734694)
(7 10 448.18666666666667)
(7 18 138.32921810699588)
(6 5 829.44)
(2 13 0.504930966469428)
(2 4 5.333333333333333)
(6 14 105.79591836734694)

8.2. f_2

(4 17 -2.4497720911755554)
(18 18 89295665425899.97)
(14 15 -5886448.888255767)
(5 1 0.15592448293623584)
(6 0 0.09047539154589039)
(2 11 -0.011618783380305042)
(3 12 -0.12606209562358486)
(15 16 -38931442.58443023)
(9 12 -1318.6359677907499)
(12 14 -181011.55860384842)
(4 15 -1.5999968706280614)
(7 3 0.7676317512752314)
(13 4 14.10577688348565)
(7 4 1.7429185241417005)
(9 10 -972.9858294278807)
(17 3 25.555036074522636)
(2 8 -0.022675969371921127)
(7 18 -1355.8618354315115)
(6 17 -150.36511151795014)

(2 3 -0.29251189747407047)
(17 19 -2711501213.613263)
(19 12 2381741.3955791583)
(9 0 0.1219744326565336)
(7 0 0.09461557702483518)
(2 2 2.718281828459045)
(8 14 -847.1487092209862)
(5 14 -8.697010469045177)
(9 2 0.5807556858098182)
(2 13 -0.008237769270189607)
(11 2 1.0412422649541293)
(6 10 -12.716588105740703)
(17 12 751887.6135417097)
(8 0 0.10499644237143123)
(12 18 -1882257.8027239882)
(2 2 2.718281828459045)
(17 6 701.4163925167654)
(3 14 -0.1365009957082285)
(12 5 24.875518175753292)
(2 11 -0.011618783380305042)
(19 5 581.8154927034864)

8.3. f_3

(7 19 0.021467462815455557)
(15 5 0.8972897520305314)
(3 12 0.034300505648069725)
(4 16 0.02618158100704072)
(5 3 0.3234657858799432)
(14 10 0.09287287780685931)
(9 17 0.02363015892994423)
(17 17 0.06919428556089095)
(17 1 2.88274645317702e+86)
(19 3 110.46198121467432)
(15 17 0.03143716981725172)
(19 8 0.0535003236864474)
(3 16 0.02618158100704072)
(10 8 0.0502589029357979)
(2 18 0.02219768929739122)
(19 15 0.11946085767729647)

(4 12 0.034300505648069725)
(4 8 0.060623809023872356)
(5 5 0.4031782747928774)
(17 17 0.03294052780160192)
(16 14 0.0877734809884482)
(5 4 0.21784251308036023)
(14 3 4.655292450035863)
(12 2 17.95581981811268)
(12 2 4978.651181453867)
(15 18 0.02586177411353612)
(10 2 1.473903445060754)
(17 8 3.109408219964699)
(9 16 0.033032040793022216)
(12 9 0.059983006928122015)
(16 17 0.023835458764299447)
(17 13 0.030687867723187132)
(8 6 0.13131681414605237)
(6 1 0.657744623479457)
(14 13 0.08003365790226269)
(7 14 0.030372418985094643)
(9 2 91.18717719247881)
(9 11 0.04724676957863885)
(8 15 0.0281155302113515)
(14 7 56.43618334181874)

8.4. $f_4(x, y) = x + y$

(7 1 8)
(18 13 31)
(9 9 18)
(14 4 18)
(10 9 19)
(18 1 19)
(16 5 21)
(0 16 16)
(19 11 30)
(10 9 19)
(14 4 18)
(12 18 30)
(3 11 14)

(10 0 10)
(17 17 34)
(2 0 2)
(17 15 32)
(15 1 16)
(2 18 20)
(4 1 5)
(13 19 32)
(5 19 24)
(14 8 22)
(9 18 27)
(16 12 28)
(3 12 15)
(15 12 27)
(0 15 15)
(13 18 31)
(9 19 28)
(7 4 11)
(19 19 38)
(14 19 33)
(6 6 12)
(8 17 25)
(9 3 12)
(14 2 16)
(3 1 4)
(13 5 18)
(12 6 18)

8.5. $f_5(x, y) = x^2 - y^2$

(6 10 -64)
(6 9 -45)
(19 9 280)
(18 19 -37)
(6 5 11)
(7 18 -275)
(2 4 -12)
(7 16 -207)
(1 5 -24)
(4 18 -308)

(0 13 -169)
(1 8 -63)
(11 14 -75)
(7 17 -240)
(0 19 -361)
(2 0 4)
(15 9 144)
(13 0 169)
(12 14 -52)
(10 11 -21)
(3 6 -27)
(1 11 -120)
(10 7 51)
(0 9 -81)
(14 19 -165)
(9 15 -144)
(4 10 -84)
(3 8 -55)
(9 1 80)
(16 19 -105)
(2 4 -12)
(17 11 168)
(14 19 -165)
(19 11 240)
(7 2 45)
(5 14 -171)
(6 14 -160)
(4 2 12)
(1 7 -48)
(2 3 -5)

8.6. $f_6(x, y) = log_{x+2}(y + 2)$

(11 3 0.6274735630753033)
(11 2 0.5404763088546395)
(17 11 0.8711164930263436)
(19 18 0.983974444213354)
(10 17 1.184929413512433)
(3 6 1.2920296742201791)
(8 13 1.1760912590556811)

(7 13 1.2324867603589635)
(13 15 1.0462189153299533)
(3 11 1.5936926411670824)
(6 7 1.0566416671474377)
(4 5 1.0860331325016919)
(1 2 1.2618595071429148)
(11 19 1.1869717539906937)
(16 19 1.0533324751067528)
(18 16 0.9648297958439285)
(4 9 1.3382908331057726)
(19 12 0.8668214419824227)
(4 18 1.6719500161730103)
(14 5 0.701838730514401)
(3 12 1.6397385131955606)
(4 5 1.0860331325016919)
(5 10 1.276989408269624)
(0 17 4.247927513443585)
(7 13 1.2324867603589635)
(12 9 0.9086181061280522)
(13 5 0.7185650207899779)
(19 12 0.8668214419824227)
(10 5 0.783091851446946)
(4 15 1.5812464746045694)
(2 0 0.5)
(11 19 1.1869717539906937)
(6 12 1.2691183073525347)
(1 5 1.7712437491614221)
(7 13 1.2324867603589635)
(4 7 1.226294385530917)
(8 17 1.2787536009528289)
(14 1 0.3962406251802891)
(14 9 0.8648579046593244)
(3 17 1.8294828004351504)

8.7. Documentación

Toda la documentación debe estar hecha en LaTeX o de lo contrario, no será recibida.

Tener presente que la ortografía resta 1 punto por cada falta y no tener bibliografía implica una nota de cero.

Como siempre, la nota del proyecto es el promedio geométrico de la documentación y la programada.

Producto	**Valor**
Portada	5 %
Resumen ejecutivo	10 %
Introducción	5 %
Marco teórico	10 %
Análisis de resultados	55 %
Conclusiones	10 %
Aprendizajes	5 %
Total	100 %

9. Estimación de tiempo

En ejecución, puede tardar en encontrar las funciones, pero en programación, debiera ser corto.

10. Aspectos Administrativos

- El trabajo debe de ser realizado en grupos de a lo sumo dos estudiantes.
- Toda la programación debe realizarse sobre Linux.
- La fecha de entrega de este proyecto es el 11 de setiembre de 2025 a la hora de clase.
- El medio de entrega será por correo electrónico al correo: edramirez@tec.ac.cr, el asunto debe ser: **IC-4700 - Lenguajes - Proyecto [Nombres de los estudiante]**. Todos los miembros del equipo deben estar copiados en el correo.
- El archivo debe tener un encabezado indicando el nombre de los autores y el detalle del programa
- Cualquier sospecha de fraude se asignará la nota mínima en el curso y se enviará una carta al expediente.